Preliminary Results

Ryan Waterman

Table of Contents

Data Preparation

The data preparation process conducted during the initial experimentation can be broken into three subsections: one-hot encoding and train-test-validation data splitting. Each of these subsections are explained, in detail, below.

One-hot encoding was performed on all categorical variables using the algorithm described below:

1. Determine the categorical and quantitative variables. This was performed using the describe() function from pandas, which only outputs columns with numerical data.

```
data = pd.read_csv('project_data.csv')

quantitative = set((data.drop('Booking_ID', axis=1).describe().columns))
categorical = set((data.drop(['Booking_ID', 'arrival_date', 'booking_status'], axis=1).columns)) - quantitative
```

2. Iterate through the categorical variable columns in the dataset, then identify the unique values for each column. For each unique value, generate a new column (named after the unique value), then generate an array with 1 at the indices of the unique value for the initial column and 0 elsewhere.

```
for cat in list(categorical):
    # Find the unique values for the column
    unique_vals = data[cat].unique()

    # For each unique value, create a new column with the value name
    for val in unique_vals:
        data[val] = np.where(data[cat].values==val, 1, 0)
```

3. Note that booking_status was omitted from the categorical columns. This is the Y variable that the model will be predicting, therefore it will remain as one column as it is encoded to 1 and 0.

```
encoded_data[encoded_data['booking_status'] == "not_canceled"] = 0
encoded_data[encoded_data['booking_status'] == "canceled"] = 1
```

The train-test-validation split selected for the entirety of the analysis was 70% train, 20% test, and 10% validation with a random seed generated by torch.seed(). The algorithm used to split the data can be found below:

```python
def _split_data(self):
    n_samples = len(self.X)
    n_test = int(0.20 * n_samples)
    n_val = int(0.10 * n_samples)

    shuffled_indices = torch.randperm(n_samples)

    train_indices = shuffled_indices[:-n_val-n_test]
    test_indices = shuffled_indices[-n_val-n_test:-n_val]
    val_indices = shuffled_indices[-n_val:]

    # Verify the indices are all unique
    train_set = set(train_indices.numpy())
    test_set = set(test_indices.numpy())
    val_set = set(val_indices.numpy())

    if not len(train_set & test_set) and not len(train_set & val_set) and not len(test_set & val_set):
        print("Train, Test, and Validation data is verified as unique.")
    else:
        print("DATA LEAK.")

    #Assign train data
    self.X_train = self.X[train_indices].to('cuda')
    self.Y_train = self.Y[train_indices].to('cuda')

    #Assign train data
    self.X_test = self.X[test_indices].to('cuda')
    self.Y_test = self.Y[test_indices].to('cuda')

    #Assign val data
    self.X_val = self.X[val_indices].to('cuda')
    self.Y_val = self.Y[val_indices].to('cuda')
```

In addition to one-hot encoding and train-test-validation split, there were omitted and feature engineered variables. As discussed in the Analytic Plan, the Booking ID variable has been omitted, as it is a unique identifier for each observation. Additionally, the Arrival Date variable has been modified to be an integer value representing only the arrival month. The intent of this feature is to capture seasonality in the data.

Model Implementation and Results

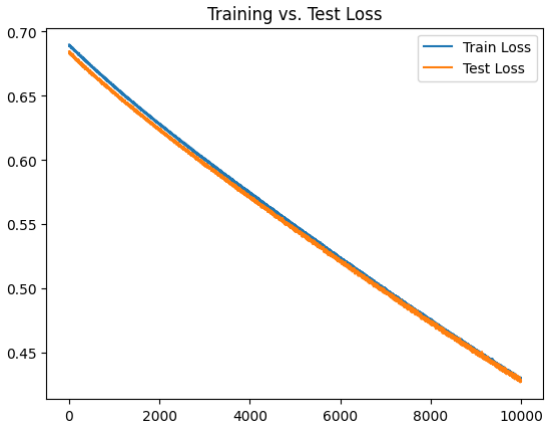
Model test was completed across 3 model architectures utilizing a custom “Model Handler” framework. The intent of developing this framework was to dramatically reduce the amount of redundant code, as well as providing a consistent set of performance metrics to evaluate the models against each other. It is important to note that each of the models were trained on the same data, determined by the code block, below:

```
torch.seed()
X = encoded_data.drop(["booking_status"], axis=1)
Y = encoded_data['booking_status'].astype(np.int64)
input_features = len(encoded_data.drop('booking_status', axis=1).columns)
labels = ['Not Canceled', 'Canceled']
```

The model architectures, as well as the Learning Curves, Confusion Matrices, ROC Curves, and AUC scores are described under each of the “**MODEL**” headers, below.

MODEL 1

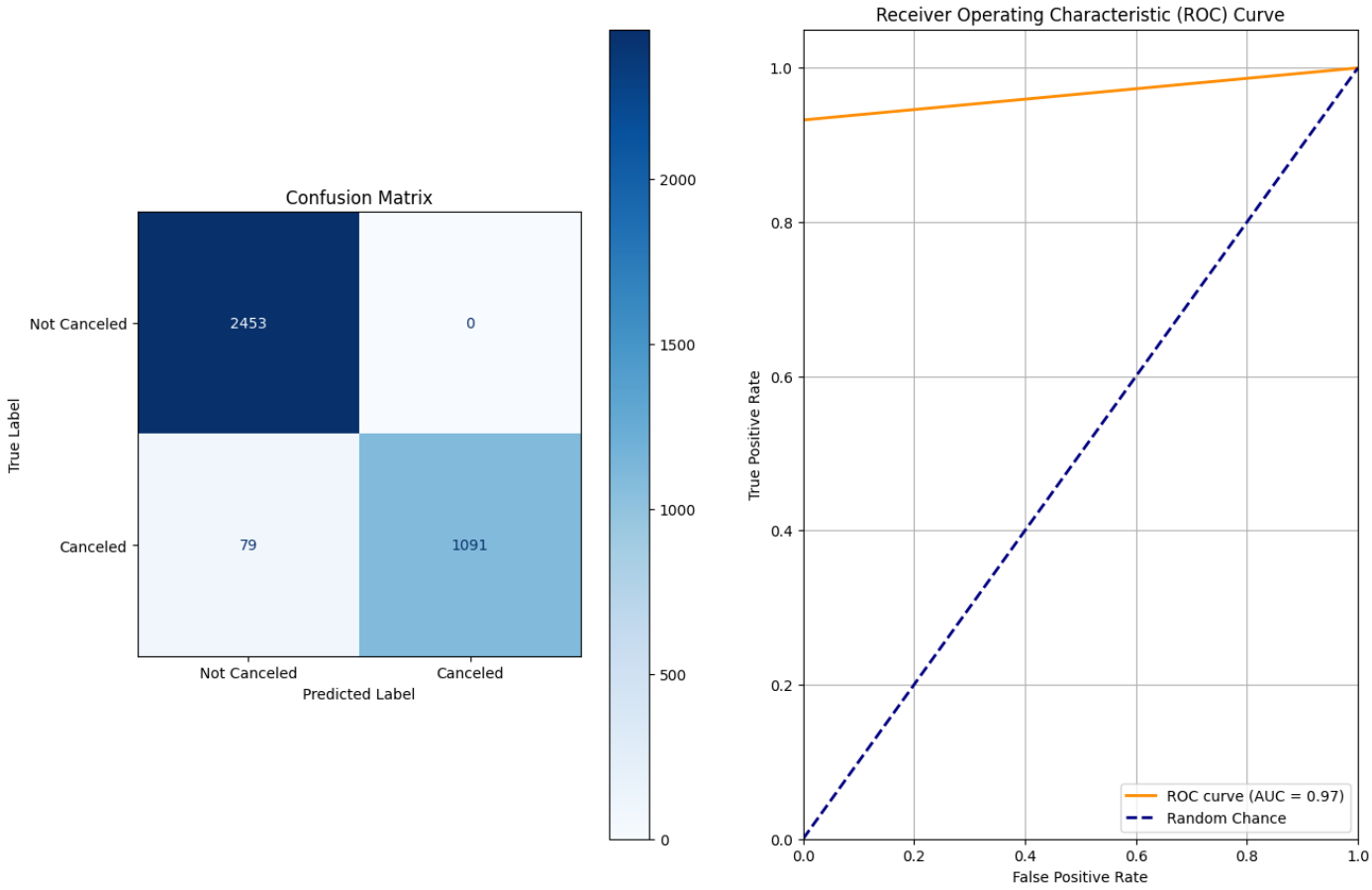
```
model_1 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=10)),
    ('dropout1', nn.Dropout(0.2)),
    ('hidden_linear2', nn.Linear(in_features=10, out_features=5)),
    ('hidden_activation', nn.Tanh()),
    ('output_linear', nn.Linear(in_features=5, out_features=2))
]))

model_1 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_1,
    optimizer = optim.SGD,
    loss_fn = nn.CrossEntropyLoss
    )

model_1.train(10000, 1e-4)
```



Model 1 Architecture and Loss Plot

Model 1 was trained using a Stochastic Gradient Descent optimizer and Cross Entropy Loss function. Cross Entropy Loss was selected as it is outputs a log loss value for each class in a classification model. In this application, there are two classes, canceled and not canceled, and the log loss of the prediction can be used to interpret the prediction probability of each class.

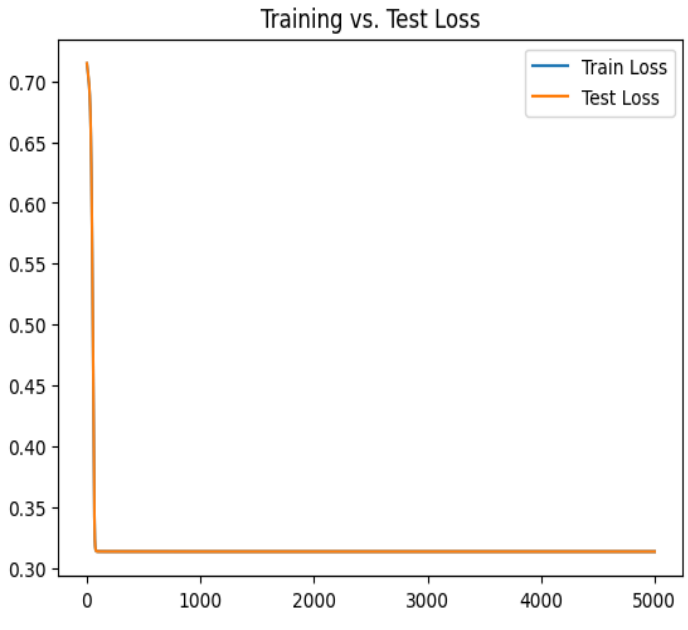
One dropout layer was included with a 20% dropout rate, as well as a Tanh activation function, which assist in adding non-linearity and addressing vanishing gradients. This is an intentionally small model, so these are the only activation/randomization functions used.



Model 1 Validation Performance Metrics

MODEL 2

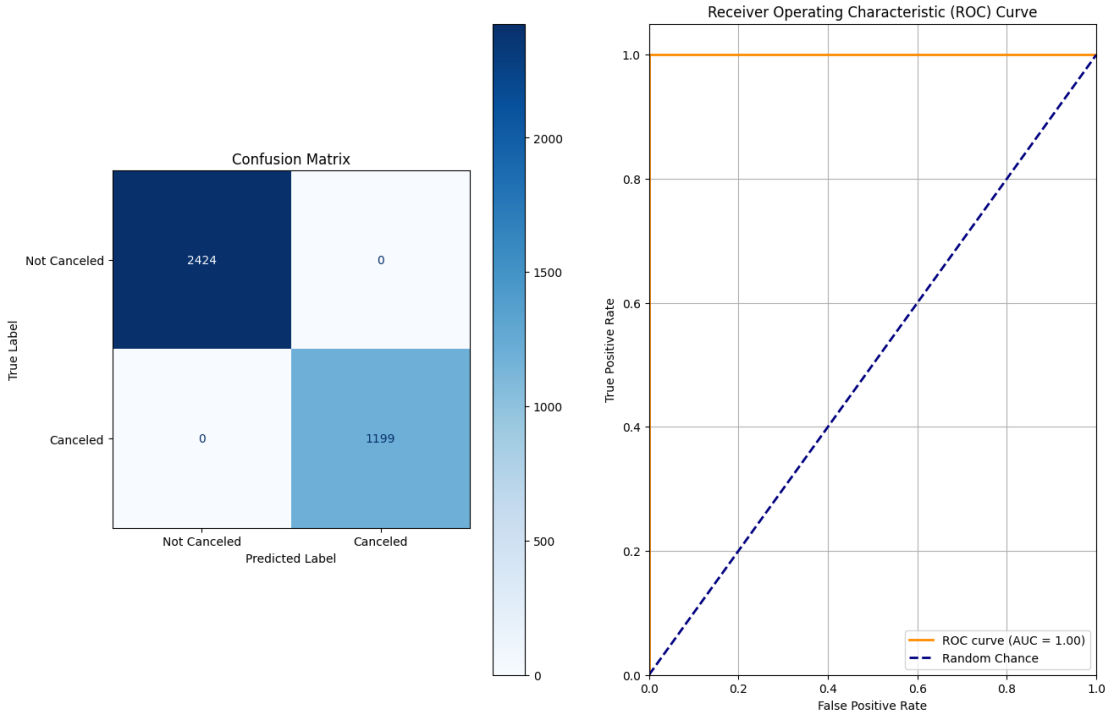
```
model_2 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=200)),
    ('hidden_linear2', nn.Linear(in_features=200, out_features=500)),
    ('relu1', nn.ReLU()),
    ('dropout1', nn.Dropout(0.2)),
    ('hidden_linear3', nn.Linear(in_features=500, out_features=400)),
    ('relu2', nn.ReLU()),
    ('dropout2', nn.Dropout(0.3)),
    ('hidden_linear4', nn.Linear(in_features=400, out_features=200)),
    ('lrelu1', nn.LeakyReLU()),
    ('dropout3', nn.Dropout(0.3)),
    ('hidden_linear6', nn.Linear(in_features=200, out_features=150)),
    ('relu3', nn.ReLU()),
    ('dropout4', nn.Dropout(0.2)),
    ('hidden_linear7', nn.Linear(in_features=150, out_features=100)),
    ('hidden_linear8', nn.Linear(in_features=100, out_features=50)),
    ('relu4', nn.ReLU()),
    ('dropout5', nn.Dropout(0.3)),
    ('hidden_linear9', nn.Linear(in_features=50, out_features=20)),
    ('lrelu2', nn.LeakyReLU()),
    ('dropout6', nn.Dropout(0.3)),
    ('hidden_linear10', nn.Linear(in_features=20, out_features=2)),
    ('output', nn.Softmax())
]))

model_2 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_2,
    optimizer = optim.AdamW,
    loss_fn = nn.CrossEntropyLoss
    )

model_2.train(5000, 1e-4)
```



Model 2 Architecture and Loss Plot

Model 2 was designed to identify the impact of scale on training and model performance. Similar to Model 1, Cross Entropy Loss was selected as the loss function, but the optimizer was changed to AdamW. The AdamW optimizer was selected because the model is larger, and the adaptive learning rate intuitively seems like a better optimization function for deeper models.

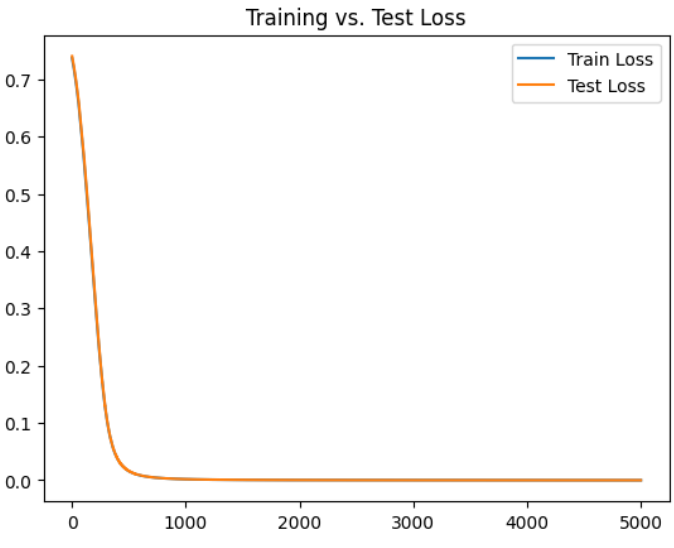
This model utilized a Softmax output layer, as well as more consistent activation-dropout layer pairing throughout. Softmax outputs a PDF for each of the classes, which simplifies the process of identifying the cancellation probability when running inference on the deployed model.



Model 2 Validation Performance Metrics

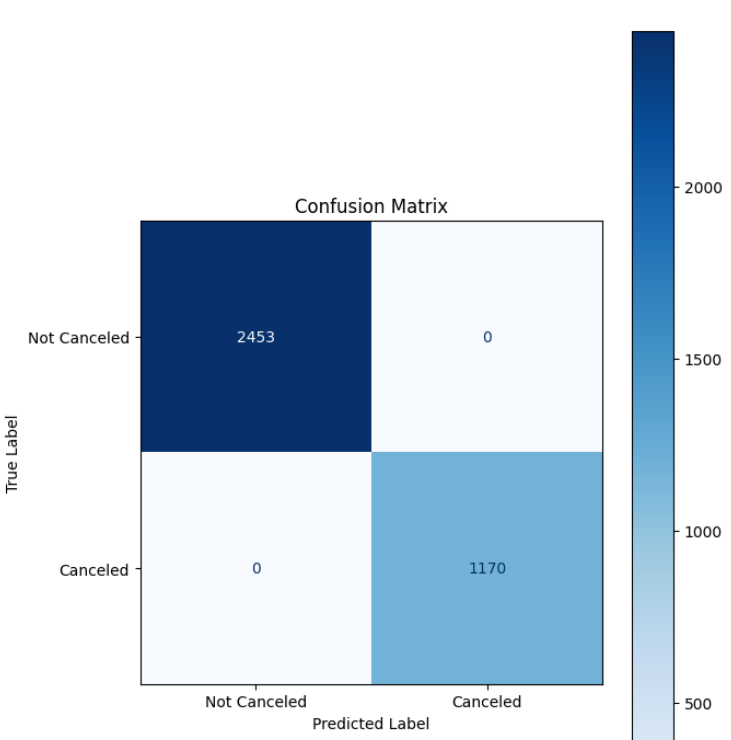
MODEL 3

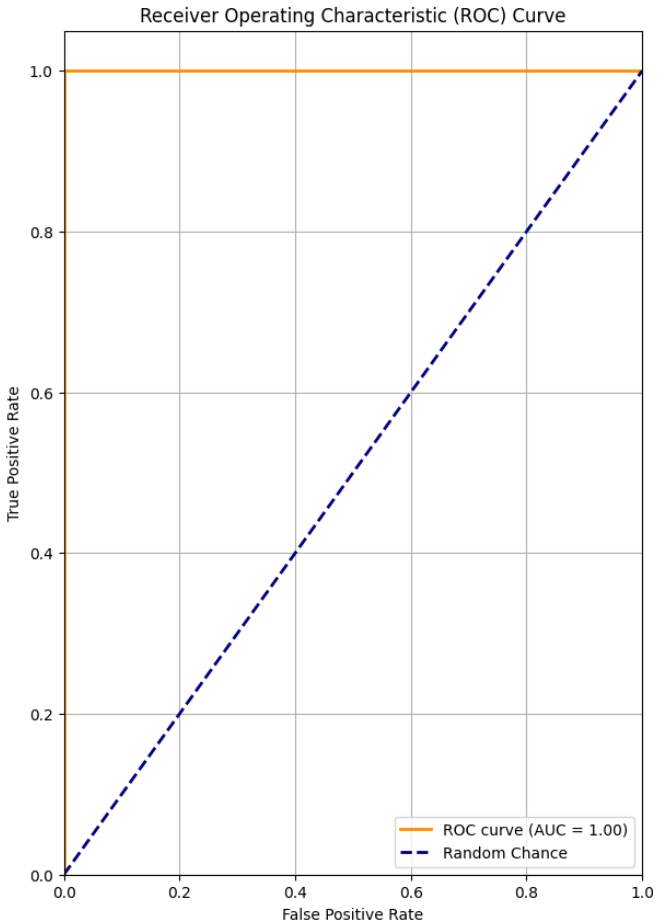
```
model_3 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=20)),
    ('hidden_linear2', nn.Linear(in_features=20, out_features=50)),
    ('dropout1', nn.Dropout(0.2)),
    ('hidden_linear3', nn.Linear(in_features=50, out_features=40)),
    ('dropout2', nn.Dropout(0.3)),
    ('hidden_linear4', nn.Linear(in_features=40, out_features=20)),
    ('hidden_activation', nn.LeakyReLU()),
    ('hidden_linear6', nn.Linear(in_features=20, out_features=2)),
    ('output', nn.LogSoftmax())
]))

model_3 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_3,
    optimizer = optim.AdamW,
    loss_fn = nn.NLLLoss
    )

model_3.train(5000, 1e-4)
```



Model 3 Architecture and Loss Plot

After identifying a dramatic increase in training time on model 2, model 3 aims to achieve similar performance without the training overhead of a much larger model. The output layer was changed to LogSoftmax to accommodate the Negative Log Likelihood Loss function, which is similar to Cross Entropy, but instead maximizes the probability of the correct class.





Model 3 Performance Metrics

Combined Performance Metrics

Model Type	Validation Accuracy	Recall	Precision	F1-Score	Overfitting?
Model 1	97.82%	93.2%	100%	0.97	No
Model 2	100%	100%	100%	1	Unclear
Model 3	100%	100%	100%	1	Unclear

Performance Metrics Across Models 1, 2, and 3

Based on the above model metrics, one could conclude that the models are overfitting to training data. When analyzing the train versus test loss plots for models 2 and 3, overfitting is quite clear, as the loss for these models plummets to 0. A counter to this notion, however, is that the validation accuracy reached 100% for both models. The picture this paints is, yes, the models have overfit to the training data, but more importantly, the data is quite linear and contains minimal, if any, outliers. The validation data is completely novel to the model, yet the overfit model can predict on this data with certainty. That said, the expected output of the model during deployment is a probability of cancellation, so there is additional work to be done to determine whether the probability of a cancelation is accurate. If the outcome of the additional analysis on the probability output points to overfitting, then additional activation and dropout layers can be introduced. Additionally, the model training can be stopped after the loss begins to flatten or reach 0.

Recommendations for Phase 3

Based on the outcome of the model evaluation phase, each of the model architectures are viable going forward. Additional features could be added by arbitrarily combining different quantitative variables with various mathematical operations to introduce variance into the data. This would introduce combinations of variables with high predictive value that may not have been explored otherwise.